

Monte Carlo simulation of the Ising model using the Metropolis Algorithm

SM Anis (Nahian)

February 2025

1 Algorithm

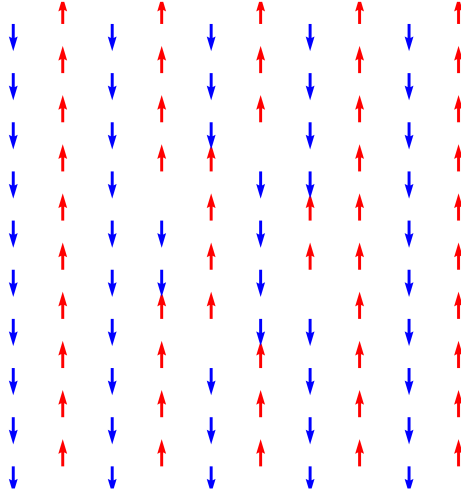


Figure 1: Two dimensional array of spins. Arrows pointing up indicate +1 spin, and arrows pointing down indicate -1 spin

For a system in thermodynamic equilibrium with reservoir in temperature T , with $\beta = \frac{1}{k_B T}$, the probability of the system to be in μ energy state is given by

$$P_\mu = \frac{1}{Z} e^{-\frac{E_\mu}{k_B T}} \quad (1)$$

$$= \frac{1}{Z} e^{-\beta E_\mu} \quad (2)$$

in canonical ensemble. The Boltzman constant $K_B = 1.38 \times 10^{-23} JK^{-1}$ is simply a conversion constant between units of energy. Now it's clear that the partition function Z can be written by summing equation (2) and using the fact that

$$\sum_\mu P_\mu = 1 \quad (3)$$

$$Z(\beta) = \sum_\mu e^{-\beta E_\mu} \quad (4)$$

it is got,

From this partition function, the average of any quantity for this system can be calculated easily. The average $\langle O \rangle$ of a physical observable O_μ is equal to

$$\langle O \rangle = \sum_{\mu} P_{\mu} O_{\mu} \quad (5)$$

$$= \frac{1}{Z} \sum_{\mu} O_{\mu} P_{\mu} \quad (6)$$

. Now, if a two-dimensional Ising model is taken with length $L=1000$, then the total number of states will be $2^{1000 \times 1000}$. Sampling this large number of states is challenging even for an excellent computer. Another fact is that an expectation value is regarded as a time average over the states that the system passes through during the period of measurement, but a system can not pass through every possible state during the measurement. Clearly, our laboratory systems are sampling only the smallest part of the sample space. However, in equation (6), all the states are considered. So clearly, we need to go for importance sampling, in which all the important samples will be generated using the Boltzmann factor as a guide. For this importance sampling, the Metropolis algorithm will be used. This algorithm will be discussed in the next section

1.1 Metropolis Algorithm

Ising model is a simple model in which magnetic dipoles or spins are placed on the sites i of a lattice. Each spin can take the value of either $+1$ or -1 , as shown in Fig. 1. If there are N sites on the lattice, then the Hamiltonian of the system is given by

$$H = -J \sum_{\langle ij \rangle} s_i s_j - B \sum_i s_i, \quad (7)$$

$\langle ij \rangle$ means the sum over all nearest neighbors. Now, after reaching equilibrium, the system must obey the detailed balance condition, which is given by,

$$\frac{P(\mu \rightarrow \nu)}{P(\nu \rightarrow \mu)} = e^{-\beta(E_{\nu} - E_{\mu})} \quad (8)$$

Here, $P(\mu \rightarrow \nu)$ and $P(\nu \rightarrow \mu)$ are the probabilities of the system transitioning from the μ state to the ν state and from the ν state to the μ state, respectively and E_{μ} and E_{ν} are the energies of the μ and ν states. Now the $P(\mu \rightarrow \nu)$ can be split in two parts $g(\mu \rightarrow \nu)$ and $A(\mu \rightarrow \nu)$ so that $P(\mu \rightarrow \nu) = g(\mu \rightarrow \nu)A(\mu \rightarrow \nu)$ here on right aide the first term is called selection probability and the second term is called acceptance ratio. In the context of the Metropolis algorithm, $g(\mu \rightarrow \nu)$ and $g(\nu \rightarrow \mu)$ are equal. So the detailed balance condition becomes

$$\frac{A(\mu \rightarrow \nu)}{A(\nu \rightarrow \mu)} = e^{-\beta(E_{\nu} - E_{\mu})} \quad (9)$$

Now, in this algorithm, the acceptance ratio is taken as

$$A(\mu \rightarrow \nu) = \begin{cases} e^{-\beta(E_{\nu} - E_{\mu})}, & \text{if } E_{\nu} - E_{\mu} > 0 \\ 1, & \text{if } E_{\nu} - E_{\mu} \leq 0 \end{cases} \quad (10)$$

According to this relation, when a change in the states lowers the energy, the change is always accepted. When it increases the energy, the change is accepted with a probability less than one. Now the Metropolis algorithm will be implemented. For this, we shall simplify the equation of change in energy. It is considered that the system is in the state $\mu = \{s_1^{\mu}, s_2^{\mu}, s_3^{\mu}, s_4^{\mu}, \dots, s_N^{\mu}\}$ then it makes a transition to a new state $\nu = \{s_1^{\nu}, s_2^{\nu}, s_3^{\nu}, s_4^{\nu}, \dots, s_N^{\nu}\}$. Here $s_k^{\mu} = -s_k^{\nu}$. If $B=0$ then equation 7 becomes $H = \sum_{\langle ij \rangle} s_i s_j$ The energy difference between two states is

$$E_{\nu} - E_{\mu} = \left(- \sum_{\langle ij \rangle} s_i^{\nu} s_j^{\nu} \right) - \left(- \sum_{\langle ij \rangle} s_i^{\mu} s_j^{\mu} \right)$$

As only one spin is flipped at a time, if the k -th spin is flipped, then the energy difference produced by that single spin flip matters because other terms in this equation will be canceled. If that k -th spin has i nearest

neighbors, then

$$= - \sum_{\langle ik \rangle} s_i^\mu (s_k^\nu - s_i^\mu)$$

Now it can be proved that $(s_k^\nu - s_i^\mu) = -2s_k^\mu$. Then

$$\begin{aligned} &= 2 \sum_{\langle ik \rangle} s_i^\mu s_k^\mu \\ &= 2s_k^\mu \left(\sum_{\langle ik \rangle} s_i^\mu \right) \end{aligned}$$

So the job has become easy. To calculate the energy difference due to a single flip, the major work is to compute the sum of the spins of the nearest neighbors of that flipped spin. In this implementation, the periodic boundary condition is used due to its simplicity. In this condition, spins on one boundary interact with spins on the opposite boundary. Now the steps are followed for writing the code

1. An initial state is generated with zero net magnetization. It means that the probability of positive and negative spins on this lattice will be equal.
2. A lattice site k is chosen.
3. The energy difference ΔE is calculated if the spin of i-th site is flipped. If ΔE is negative, then the flip is accepted. If ΔE is positive, then a random number r is generated such that $0 < r < 1$. If r is less than $e^{-\beta \Delta E}$, then the flip is accepted; otherwise, it is left as it was.
4. Step 3 is repeated for all sites

After completing the above steps for each site, a Monte Carlo step is completed. This Monte Carlo step has to be repeated over and over until the system reaches its equilibrium.

The net magnetization for each step would be $M = \sum_i s_i$, but the magnetization per site is of interest, and it is defined as $m = \frac{1}{N} \sum_i s_i$ where N is the total number of sites

2 Code implementation in Python

2.1 Code for magnetization per site vs Monte Carlo time and spins state diagram

```

1 #Monte Carlo simulation of Ising model using Metropolis algorithm
2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import random
6 from time import perf_counter
7
8 N=int(input("Enter the length of the square lattice: "))
9 upspinprobab=float(input("Enter the probability of up spin in inital state: "))
10 ite=int(input("Enter the number of total Monte Carlo Steps: "))
11 def f1(beta):
12     #Initail state sampling
13     np.random.seed(7498)
14     initial_sam=np.random.rand((N+2),(N+2)) #N+2 is taken due to implement periodic boundary
15     condition
16     for i in range(1,N+1):
17         for j in range(1,N+1):
18             if initial_sam[i,j]<upspinprobab:
19                 initial_sam[i,j]=1
20             elif initial_sam[i,j]>=upspinprobab:
21                 initial_sam[i,j]=-1
22     print(initial_sam)
23     #Entering the boundsary values
24     for l in range(1,N+1,1):
25         initial_sam[l,0]=initial_sam[l,N]
```

```

26     initial_sam[1,N+1]=initial_sam[1,1]
27     initial_sam[0,1]=initial_sam[N,1]
28     initial_sam[N+1,1]=initial_sam[1,1]
29
30     initial_sam[0,0]=0
31     initial_sam[0,(N+1)]=0
32     initial_sam[(N+1),0]=0
33     initial_sam[(N+1),(N+1)]=0
34
35     print(initial_sam)
36
37     Mo=(abs(np.sum(initial_sam)))*1/(N*N)
38     print("initial magnetization is: ",Mo)
39
40     #calculation of initial internal energy
41     E=0
42
43     for v in range(1,N+1):
44         for b in range(1,N+1):
45             E=E+(-initial_sam[v,b]*(initial_sam[v+1,b]+initial_sam[v-1,b]+initial_sam[v,b+1]+
46                 initial_sam[v,b-1]))
47     E=E/2
48     Eo=(E*(1/(N*N)))
49
50
51
52
53
54
55
56
57     #Calculating the two values of exponentials
58     A8=np.exp(-beta*8)
59     A4=np.exp(-beta*4)
60
61     #Starting the Monte Carlo simulation
62
63     random.seed(9874) #Setting up the seed to be used in the following random library
64     t=0
65     t_list=[0]
66     energy_list=[Eo]
67     Magnetization_list=[Mo]
68     for c in range(0,ite,1):#10000 is the number of iteration
69         t=t+1
70         for h in range(1,(N+1),1):
71             for k in range(1,(N+1),1):
72                 delta_E=2*(initial_sam[h,k]*(initial_sam[h+1,k]+initial_sam[h-1,k]+initial_sam[h,k+1]+
73                     initial_sam[h,k-1]))
74                 #print('delta_E:',delta_E)
75                 if delta_E<=0:
76                     initial_sam[h,k]=-initial_sam[h,k]
77                 elif delta_E==4:
78                     prob=A4
79                     r=random.random()
80                     if r<prob:
81                         initial_sam[h,k]=-initial_sam[h,k]
82                     else:
83                         initial_sam[h,k]=initial_sam[h,k]
84
85                 elif delta_E==8:
86                     prob=A8
87                     r=random.random()
88                     if r<prob:
89                         initial_sam[h,k]=-initial_sam[h,k]
90                     else:
91                         initial_sam[h,k]=initial_sam[h,k]
92
93     #calculation of internal energy
94     E=0
95
96     for q in range(1,N+1):
97         for w in range(1,N+1):

```

```

97     E=E+(-initial_sam[q,w]*(initial_sam[q+1,w]+initial_sam[q-1,w]+initial_sam[q,w+1]+
98         initial_sam[q,w-1]))
99     E=E/2
100     energy_list.append(E*(1/(N*N)))
101     M=(abs(np.sum(initial_sam)))*1/(N*N)
102     Magnetization_list.append(M)
103     t_list.append(t)
104
105     print(initial_sam)
106
107
108
109
110     #plt.imshow(initial_sam, cmap="gray", vmin=-1, vmax=+1)
111     #plt.savefig("spin.png", dpi=600)
112     #plt.show()
113     M=(abs(np.sum(initial_sam)))*1/(N*N)
114     plt.figure()
115     print("final magnetization is: ",M)
116     plt.plot(t_list,Magnetization_list,label='Magnetization per site, '+'LxL='+str(N)+'X'+str(N)
117         +', '+'beta='+str(beta))
117     plt.plot(t_list,energy_list,label='Energy per site, '+'LxL='+str(N)+'X'+str(N)+'+', '+'beta='+
118         str(beta))
118     plt.xlabel("Monte Carlo time")
119     #plt.ylabel("Magnetization per site and Energy per site")
120     plt.legend()
121     #plt.show()
122     plt.savefig("magnetization and energy vs time.png", dpi=600)
123     plt.figure()
124     #2d picture plot
125
126     plt.imshow(initial_sam, cmap="gray", vmin=-1, vmax=+1)
127     plt.savefig("spin distribution.png", dpi=600)
128     plt.show()
129
130
131     b=float(input('Enter the value of beta'))
132     t1_start = perf_counter()
133
134     f1(b)
135
136
137     t1_stop = perf_counter()
138
139     print("Elapsed time during the whole program in seconds:", t1_stop-t1_start)

```

3 Results

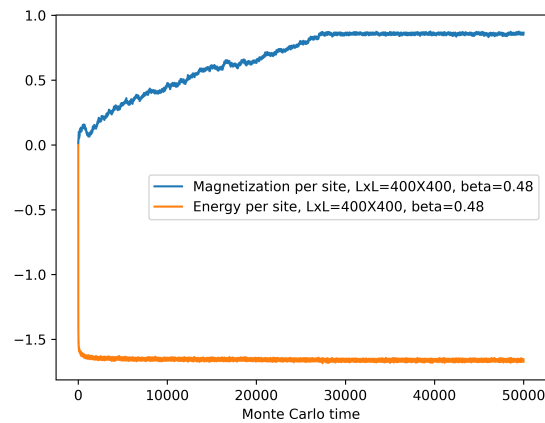
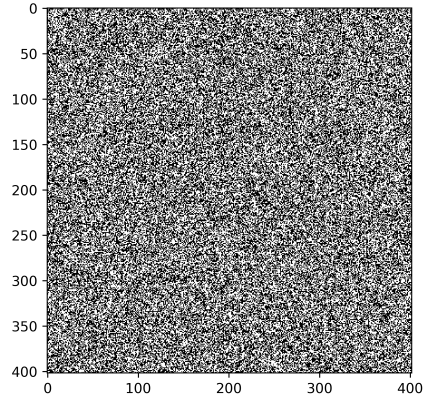
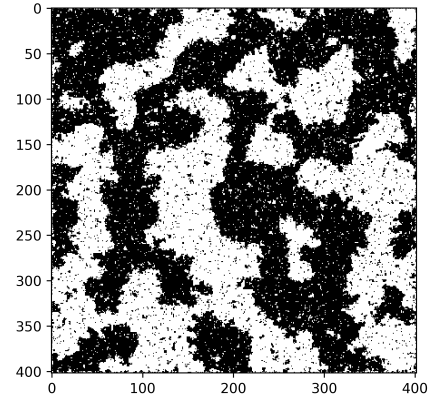


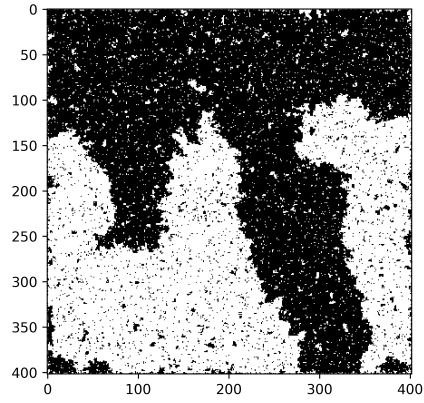
Figure 2: Magnetization per cite and energy per cite vs Monte Carlo steps



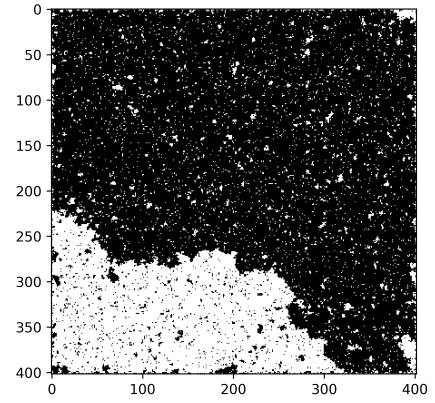
(a)



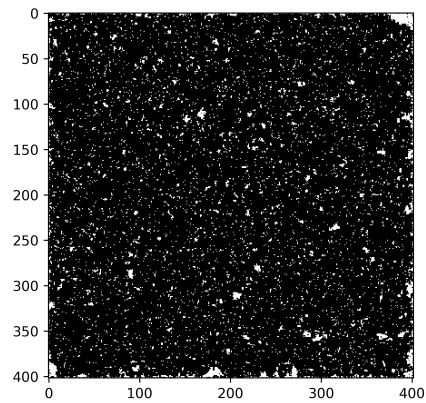
(b)



(c)



(d)



(e)

Figure 3: Spin configuration for the Ising model with $L=400, \beta = .48$ after 0,100,1000,10000,50000 Monte Carlo steps respectively

References

1. K. Anagnostopoulos, *Computational Physics: An Introduction*, University of Cambridge, Cambridge, UK.
2. M. E. J. Newman and G. T. Barkema, *Monte Carlo Methods in Statistical Physics*, Oxford University Press, Oxford, UK.